UNIT 5 · TOPIC 5.3

Protecting Stored Data with Cryptography

Quiz, Answer Key

Teacher answer key, 18 questions, 1 point each. The correct answer is highlighted; the explanation and distractor notes follow.

1. Which of the following is an example of a security control that protects data at rest?

- **● A. Full-disk encryption applied to a laptop hard drive**
- ○ B. Firewall rules that filter inbound network traffic
- ○ C. An intrusion detection system that monitors network packets
- ○ D. A VPN tunnel established between two branch offices

Correct answer: A

Why: Full-disk encryption protects data at rest by rendering the stored contents unreadable without the decryption key. Firewall rules and IDS protect data in transit or detect network-layer events; a VPN protects data in motion between sites. None of the other options act on data while it is stored on a device.

*Why the others are wrong: **B**, Students who conflate "security control" generically with data protection may select this; firewalls operate at the network boundary, not on stored data. **C**, IDS is a detective control for network traffic, not a protective control for stored files. **D**, VPNs protect data in transit, not at rest, a common conceptual conflation of the two threat categories.*

2. Which of the following best describes the defining characteristic of a symmetric encryption algorithm?

- ○ A. It uses one key to encrypt data and a mathematically related but different key to decrypt it
- ○ B. It produces a fixed-length digest from input data and cannot be reversed
- **● C. It uses the same key to both encrypt and decrypt data**
- ○ D. It distributes encrypted data across multiple servers so no single server holds a complete copy

Correct answer: C

Why: Symmetric encryption algorithms, such as AES and 3DES, use a single shared secret key for both the encryption and decryption operations. Option A describes asymmetric encryption (e.g., RSA). Option B describes a cryptographic hash function. Option D describes a secret-sharing scheme, which is a separate concept.

*Why the others are wrong: **A**, This is the defining property of asymmetric (public-key) encryption; students who have encountered RSA without anchoring the symmetric/asymmetric distinction may select this. **B**, Students who conflate hashing with encryption, both transform data, frequently select this. **D**, Plausible to students who have encountered distributed storage or RAID concepts alongside encryption.*

3. A security analyst wants to verify that a stored configuration file has not been altered since it was last saved by an authorized administrator. Which cryptographic control is most appropriate for this specific purpose?

- ○ A. AES-256 symmetric encryption of the file
- ○ B. A digital certificate issued to the administrator
- **● C. A cryptographic hash of the file stored separately in a secure location**
- ○ D. A symmetric decryption key held by the administrator

Correct answer: C

Why: A cryptographic hash function produces a fixed-length digest of the file's contents. Any subsequent modification to the file, however small, changes its hash value. By comparing the stored hash against a freshly computed hash, an analyst can detect unauthorized changes. Encryption provides confidentiality, not integrity verification. A digital certificate authenticates identity. A decryption key alone does not verify file integrity.

*Why the others are wrong: **A**, AES-256 would hide the file's contents but would not in isolation tell the analyst whether the contents were tampered with before encryption. **B**, Certificates authenticate the identity of a person or system; they do not continuously monitor file integrity. **D**, Holding a key is a component of access control, not file integrity checking.*

4. A company stores customer payment records on an internal server. An employee's laptop containing a local copy of these records is stolen from a vehicle. The IT team confirms the laptop's hard drive was encrypted with AES-256. Which statement best explains how this control mitigates the risk of data exposure?

○ A. AES-256 prevents the thief from connecting the laptop to any external network, isolating the data

○ B. The encryption ensures the data was backed up to a secure server immediately before the theft occurred

○ C. AES-256 encryption automatically triggers a remote data wipe once the laptop is reported stolen

● D. Without the correct decryption key, the ciphertext on the stolen drive is computationally infeasible to recover in any useful timeframe

Correct answer: D

Why: Full-disk encryption converts all stored data into ciphertext. An adversary who physically possesses the drive but does not hold the key cannot read the plaintext contents; brute-forcing AES-256 is computationally infeasible with current technology. The other options describe features of network access control, mobile device management (remote wipe), and backup systems respectively, distinct controls that encryption alone does not provide.

*Why the others are wrong: **A**, Students who confuse encryption with network-layer controls may believe encryption also restricts connectivity. **B**, This reflects a misconception that encryption involves data replication or synchronization. **C**, Remote wipe is a separate MDM capability; students who have seen both controls together may conflate them.*

5. A database administrator hashes all stored user passwords using SHA-256. A security auditor inspects the database and finds that two user accounts with the same password produce identical hash values. Which additional control would eliminate this vulnerability, and why does it work?

● A. Add a unique, randomly generated salt to each password before hashing, so identical passwords produce different hash outputs

○ B. Encrypt each stored hash value with AES-256 to prevent the auditor from reading the database

○ C. Replace SHA-256 with AES-256, since AES is a stronger algorithm for storing passwords

○ D. Store the original plaintext passwords alongside their hashes to allow the administrator to verify correctness manually

Correct answer: A

Why: A salt is a random value appended to a password before the hash is computed. Because each account gets a unique salt, two users with the same password will produce different hash values. This also defeats precomputed rainbow-table attacks. Encrypting the hash values (B) does not solve the duplicate-hash problem. AES (C) is a symmetric encryption algorithm, not a suitable function for one-way password storage. Storing plaintext (D) eliminates all benefit of hashing.

*Why the others are wrong: **B**, Students who know encryption is "stronger" than hashing may assume adding encryption on top solves all hashing weaknesses. **C**, Confusion between the roles of encryption (reversible) and hashing (one-way) for credential storage is a persistent misconception. **D**, This reflects a fundamental misunderstanding of why passwords are hashed in the first place.*

6. A developer must choose between AES-128 and AES-256 for encrypting sensitive health records. Which option provides stronger security, and which property of the algorithm accounts for the difference?

○ A. AES-128 is stronger because a shorter key is processed through more internal rounds, compressing the output more securely

● B. AES-256 is stronger because its longer key exponentially increases the number of possible key values, making brute-force attacks significantly harder

○ C. AES-128 and AES-256 are equally secure because both use the same 128-bit block size for data processing

○ D. AES-256 is stronger because it stores twice as many data blocks per operation, reducing the number of encryption passes needed

Correct answer: B

Why: AES-128 uses a 128-bit key (2^128 possible values) while AES-256 uses a 256-bit key (2^256 possible values). The key space of AES-256 is astronomically larger, making exhaustive key search far more computationally expensive. Both variants

use the same 128-bit block size, so block size is not the distinguishing factor. AES-128 uses 10 rounds and AES-256 uses 14 rounds, a slight reversal of the claim in option A. Option D confuses key length with block throughput.

*Why the others are wrong: **A**, Students who know AES-128 has fewer rounds than AES-256 may incorrectly infer that fewer rounds implies tighter compression and thus more security. **C**, Block size is a visible shared property; students who focus on block size rather than key size frequently select this. **D**, Students who interpret "256" as a data-volume measure rather than a key-length measure may reason this way.*

7. A data classification policy requires that files labelled "Confidential" satisfy two independent security requirements: (1) their contents must be unreadable if the storage media is seized, and (2) any unauthorised modification to the file must be detectable before the file is used. Which combination of cryptographic controls best satisfies both requirements simultaneously?

○ A. Full-disk encryption with AES-256 only

○ B. SHA-256 hashing of each file, with hashes stored in a secure registry

● C. AES-256 encryption of each file combined with a SHA-256 hash of the plaintext stored in a separate access-controlled registry

○ D. Triple-DES encryption applied in two passes to each file

Correct answer: C

Why: Requirement (1), confidentiality, is addressed by AES-256 encryption, which renders ciphertext unreadable without the key. Requirement (2), integrity verification, is addressed by the SHA-256 hash of the plaintext: before using a decrypted file, the system recomputes its hash and compares it to the stored value; any tampering produces a hash mismatch. Option A satisfies only (1). Option B satisfies only (2). Option D addresses neither requirement adequately: 3DES is considered outdated and provides no integrity mechanism.

*Why the others are wrong: **A**, Students who read requirement (1) first and stop may select the first valid control without considering requirement (2). **B**, Students who focus on requirement (2) may select a hashing-only approach without recognising it provides no confidentiality. **D**, Students who associate "stronger encryption" with applying an algorithm multiple times may select 3DES applied twice, not recognising that it does not address integrity.*

8. An organisation uses AES in ECB (Electronic Codebook) mode to encrypt a large bitmap image file for archival storage. A forensic analyst examining the encrypted file observes that regions of the ciphertext correspond visually to structural outlines of the original image. Which evaluation of this implementation is most accurate?

○ A. ECB mode is secure for image files because each block is encrypted independently, preventing any one block from leaking information about neighbouring blocks

● B. ECB mode is insecure for this use case because identical plaintext blocks produce identical ciphertext blocks, allowing patterns in the data to remain visible in the encrypted output

○ C. The pattern leakage indicates that the key length was insufficient for the size of the image file being encrypted

○ D. The vulnerability exists because AES operates as a hash function on binary data and cannot fully conceal structural information in large files

Correct answer: B

Why: In ECB mode, each 128-bit block of plaintext is encrypted independently with the same key. This means two identical plaintext blocks always produce the same ciphertext block. In structured data such as images with large uniform regions, this property allows an observer to see the same patterns in the ciphertext as in the original, completely undermining confidentiality. The key length is not the cause of this failure, ECB's deterministic block-level encryption is. AES is a block cipher, not a hash function.

*Why the others are wrong: **A**, This is a direct misreading of ECB's "independence" property, students may reason that independence between blocks prevents leakage, when in fact it is the source of the leakage. **C**, Key length determines brute-force resistance, not pattern leakage; students who know "shorter keys are weaker" may misattribute the cause. **D**, AES is a symmetric block cipher; students who conflate hashing and encryption may reason along these lines.*

9. A development team stores encrypted configuration files containing API keys on a shared internal server. They implement AES-128-CBC and embed the encryption key as a string constant in the application's source code,

which is hosted in a public GitHub repository. Evaluate the security posture of this implementation and identify the primary point of failure.

○ A. The primary weakness is AES-128; upgrading to AES-256 would make the implementation secure

○ B. The primary weakness is the use of CBC mode; switching to ECB would eliminate the vulnerability introduced by IV management

○ C. The implementation is secure because the key is embedded in compiled application code and cannot be extracted through the repository alone

● D. The primary weakness is the hard-coded key committed to a public repository; an attacker who clones the repository can decrypt every protected file regardless of algorithm strength

Correct answer: D

Why: The cryptographic algorithm (AES-128-CBC) may be technically sound, but it is completely negated by exposing the secret key in a public repository. Any person with repository access, which is unrestricted, can retrieve the key and decrypt all protected files. Upgrading the algorithm (A) does not address key exposure. ECB (B) is less secure than CBC and introduces pattern leakage. Option C reflects a misunderstanding: source code in a repository is plaintext-readable and is not compiled at rest in the repository.

*Why the others are wrong: **A**, Students who associate "stronger" exclusively with algorithm selection may focus on upgrading AES-128 to AES-256, ignoring the key management failure. **B**, Students who have learned to avoid IV misuse in CBC mode may over-index on IV-related concerns and select the weaker ECB alternative. **C**, This is a common real-world misconception: developers incorrectly believe that compiled binaries or obfuscation protect embedded secrets, which does not apply here since the key is in readable source code.*

10. When implementing symmetric file encryption, which item must be securely stored and protected to allow authorised decryption of the file in the future?

○ A. The plaintext version of the file, archived before encryption

○ B. The name of the encryption algorithm used

● C. The symmetric key used during the encryption operation

○ D. The original file size in bytes before encryption was applied

Correct answer: C

Why: In symmetric encryption, the same key is required for both encryption and decryption. If the key is lost or exposed, authorised decryption becomes impossible or the data is compromised respectively. The algorithm name is typically public (Kerckhoffs's principle). Archiving plaintext defeats the purpose of encryption. File size metadata is not required for decryption.

*Why the others are wrong: **A**, Students who think "you need the original to decrypt" may select this, conflating decryption with having the original. **B**, Students who over-index on algorithm secrecy, conflating key secrecy with algorithm secrecy, may select this. **D**, Some students believe metadata like file size is a decryption prerequisite because padding operations involve length.*

11. A technician encrypts a sensitive report file using AES-256 from the command line. After the operation completes, the original unencrypted file remains on disk alongside the new encrypted version. What additional implementation step is required to complete data protection?

● A. Securely delete the original plaintext file so that it cannot be recovered from disk

○ B. Re-encrypt the original file a second time using a different algorithm to add a second layer of protection

○ C. Share the encryption key with all team members to ensure redundant access to the data

○ D. Convert the encrypted file to a compressed archive format to improve storage efficiency

Correct answer: A

Why: Encrypting a file and leaving the original plaintext alongside it does not protect the data, an attacker who accesses the storage medium can simply read the unencrypted original. The original must be securely deleted (using a tool that overwrites the file's data blocks, not just removes the directory entry) to complete the protection. Distributing the key

increases exposure. Re-encrypting with a different algorithm does not remove the plaintext. Compression is unrelated to security.

*Why the others are wrong: **B**, Students who think "more encryption equals more security" may select this, without recognising that the plaintext copy remains accessible. **C**, Students who think redundancy is always a security benefit may select this, not recognising that key distribution expands the attack surface. **D**, Students who associate compression with data-hiding may conflate compression with security.*

12. Which of the following most accurately describes what happens during a symmetric key encryption operation on a file?

○ A. The file is split into two halves; each half is encrypted independently using a different portion of the key

○ B. The file is uploaded to a cloud provider, which applies server-side encryption on the organisation's behalf

○ C. The file is compressed into a smaller format and transmitted over a secure channel to a remote server

● D. The file's contents are processed by the encryption algorithm together with the secret key to produce a ciphertext output

Correct answer: D

Why: Symmetric encryption takes the plaintext input and a secret key, then runs both through the algorithm (e.g., AES) to produce ciphertext. The process is deterministic given the same key and plaintext. Splitting and using different key portions (A) is a rough description of certain stream cipher or key scheduling details, not the operation as a whole. Options C and D describe transmission and cloud storage, not a local encryption operation.

*Why the others are wrong: **A**, Students who vaguely recall that block ciphers divide data into blocks may over-generalise and describe the split as using different key halves. **B**, Cloud encryption is a real pattern; students familiar with S3 server-side encryption may default to this framing. **C**, Students who associate data protection with "sending it somewhere safe" may conflate encryption with secure transmission or remote storage.*

13. A developer implements AES-CBC encryption for a file storage application. She generates a random Initialization Vector (IV) for the first file encrypted, then reuses that same IV for every subsequent file encrypted with the same key. Which specific security problem does IV reuse introduce in CBC mode?

○ A. Reusing an IV causes the encryption process to run faster, which creates a timing channel that an attacker can exploit

● B. Reusing an IV with the same key means that an attacker who knows the plaintext of one file may be able to infer information about the plaintext of other files encrypted with the same IV and key pair

○ C. The IV is a secret value; reusing it publicly exposes it to any observer of the ciphertext, directly compromising the encryption key

○ D. Reusing an IV causes decryption to fail, so the encrypted files cannot be recovered at all

Correct answer: B

Why: In CBC mode, the IV is XOR-ed with the first plaintext block before encryption. If the same IV and key are used for two different files, the relationship between the two ciphertexts leaks information about the relationship between the two plaintexts, an attacker with a known plaintext for one file can exploit this to gain partial information about another. The IV is not secret (it is commonly stored alongside ciphertext), but it must be unique per encryption operation. IV reuse does not cause decryption to fail nor does it directly expose the key.

*Why the others are wrong: **A**, Students who know that implementation shortcuts sometimes introduce timing channels may select this, but IV reuse does not directly create a timing vulnerability. **C**, Students who conflate IV and key secrecy may reason that if the IV is exposed, the key is too, but the IV is not secret and its exposure alone does not compromise the key. **D**, This is a common assumption: that implementation errors break decryption entirely. IV reuse produces incorrect security, not decryption failure.*

14. An organisation needs to encrypt a large database of structured records at rest and must choose between AES-ECB mode and AES-GCM mode. Apply your understanding of encryption modes to determine which is more appropriate and justify the selection.

○ A. AES-ECB is more appropriate; its simpler implementation provides the same confidentiality guarantees as GCM without the overhead of managing nonces

○ B. AES-GCM is more appropriate only when data is transmitted over a network; for data stored at rest, AES-ECB provides equivalent protection

○ C. AES-ECB is more appropriate; GCM mode cannot process files larger than 64 KB without splitting them into separate encryption streams

● D. AES-GCM is more appropriate; it provides both confidentiality and authenticated integrity in a single operation, and avoids the pattern-leakage problem that affects ECB mode

Correct answer: D

Why: AES-GCM (Galois/Counter Mode) is an authenticated encryption mode: it provides confidentiality (like all AES modes) and also generates an authentication tag that detects any tampering with the ciphertext. Critically, GCM does not suffer from the pattern-leakage vulnerability of ECB, where identical plaintext blocks produce identical ciphertext blocks. There is no 64 KB size restriction on GCM. The in-transit vs. at-rest distinction does not determine whether authenticated encryption is appropriate, integrity protection is valuable in both contexts.

*Why the others are wrong: **A**, Students who equate simplicity of implementation with security adequacy, or who have not studied ECB's pattern-leakage weakness, may select this. **B**, Students who partition security concerns strictly into "transit" vs. "rest" categories may reason that authentication is only needed during transmission. **C**, No such size restriction exists; students may confuse GCM's nonce size (96 bits) with a data processing limit.*

15. A security engineer uses an AI-powered code assistant to generate a Python function that encrypts user files with AES-128-CBC. The AI-generated code is functional but initialises the IV as a fixed sequence of 16 zero bytes for every encryption call. Which implementation detail must the engineer correct before deployment, and why?

○ A. Replace the IV with the user's hashed password to make the encryption personalised to each user

○ B. Store the fixed IV in an encrypted configuration file alongside the key so that it is protected from disclosure

● C. Replace the fixed IV with a securely generated random IV for each individual encryption operation, ensuring ciphertext unpredictability across calls

○ D. No correction is needed; the value of the IV does not affect the confidentiality provided by AES-CBC

Correct answer: C

Why: A fixed, predictable IV (such as all zeros) used across multiple encryption operations with the same key undermines semantic security in CBC mode. If an attacker can predict the IV before a message is encrypted, they can craft chosen-plaintext attacks. The IV must be randomly generated per operation and is typically stored alongside the ciphertext (it is not secret). Using a user's hashed password as the IV is insecure because it may be predictable and reused if the user's password does not change. Encrypting the IV alongside the key does not make a static IV acceptable.

*Why the others are wrong: **A**, Students who want to make encryption "user-specific" may reach for the password as an obvious per-user value, without recognising that predictable IVs are the vulnerability. **B**, Students who know the IV need not be secret may conclude that protecting it with encryption is a valid resolution to any IV-related concern. **D**, Students who have not studied IV-based attacks may believe the IV is merely a technical formality with no security consequence.*

16. A development team documents the following implementation steps for their file encryption module:

Step 1, Derive the AES-256 key: key = MD5(user_password)
Step 2, Encrypt the file using AES-256-CBC with the derived key
Step 3, Append the IV to the beginning of the ciphertext output file
Step 4, Overwrite and delete the original plaintext file

Evaluate the implementation and identify the most critical security flaw.

● A. Step 1 is flawed; MD5 is a fast, cryptographically broken hash function that is unsuitable as a key derivation function, a purpose-built KDF such as PBKDF2 or Argon2 with appropriate iteration count should replace it

○ B. Step 3 is flawed; storing the IV in the output file exposes it to any attacker who accesses the ciphertext

○ C. Step 4 is flawed; the plaintext file should be archived in a separate secure location before deletion in case the key is lost

○ D. Step 2 is flawed; AES-256-CBC is not approved for file encryption by current NIST guidelines and must be replaced with a compliant algorithm

Correct answer: A

Why: MD5 was designed for speed and is cryptographically broken for collision resistance. Using MD5 to derive a key from a password is critically insecure: it is fast enough that an attacker can run billions of guesses per second using GPU acceleration. A proper password-based KDF (PBKDF2, bcrypt, scrypt, Argon2) is designed to be deliberately slow and includes an iteration count or work factor that scales with hardware improvements. Step 3 is standard practice, IV need not be secret. Step 4 is correct and necessary. AES-256-CBC remains an approved cipher under NIST SP 800-38A.

*Why the others are wrong: **B**, Students who have learned that IVs should not be reused or shared may over-extend that principle and conclude that storing the IV at all is a flaw. **C**, Students who have been taught about backup and recovery may apply a data-availability principle inappropriately to a data-confidentiality context. **D**, Students who know that some algorithms become deprecated may assume AES-CBC is one of them; AES-256-CBC is still NIST-approved for symmetric encryption (though GCM is preferred for authenticated encryption).*

17. A medical records application must satisfy three requirements for data stored on its servers: (a) encrypted data cannot be decrypted without authorised credentials, (b) any tampering with ciphertext stored on disk is detectable before the data is used, and (c) all encryption and decryption events must be auditable. Design the mitigation strategy that most completely addresses all three requirements, and identify which component satisfies each.

○ A. Use AES-256-ECB for (a), store a separate SHA-256 checksum of each encrypted file for (b), and enable OS filesystem audit logging for (c)

● B. Use AES-256-GCM for (a) and (b) simultaneously, GCM's authentication tag detects ciphertext tampering, and implement application-level structured logging of all encrypt/decrypt calls with timestamps and operator identity for (c)

○ C. Use SHA-256 hashing for (a), AES-128-CBC for (b), and store all audit logs encrypted with the same key used for the records for (c)

○ D. Use AES-256-CBC for (a), configure firewall rules to block unauthorised write access to the database for (b), and conduct quarterly manual reviews of access logs for (c)

Correct answer: B

Why: AES-256-GCM is an authenticated encryption scheme: the GCM authentication tag is computed over the ciphertext and fails verification if the ciphertext has been altered, satisfying both (a) and (b) with a single mechanism. Application-level logging of encrypt/decrypt operations, including timestamps, user identities, and file identifiers, satisfies (c). ECB (A) satisfies (a) but introduces pattern leakage and does not natively detect tampering. SHA-256 (C) provides no confidentiality. Firewall rules (D) prevent new unauthorised writes but cannot detect tampering of existing stored data. Manual review (D) is not a technical control and introduces unacceptable latency.

*Why the others are wrong: **A**, Students who know that checksums detect modification may select this, but ECB's pattern leakage makes (a) insufficient, and the checksum of the ciphertext (not plaintext) detects only certain forms of tampering. **C**, This reflects a fundamental role-reversal: SHA-256 provides no confidentiality, so assigning it to requirement (a) is incorrect. **D**, Firewalls and access controls are preventive for future writes but cannot satisfy integrity detection for already-stored data that may have been tampered with.*

18. An organisation deploys an AI-assisted security tool that automatically recommends encryption settings for new file storage systems. For a new internal archive containing financial records, the AI recommends: AES-128-CBC with an IV derived deterministically from the system hostname, and the encryption key stored as a plaintext environment variable readable by all application processes on the host. A security engineer must evaluate the AI's recommendation and implement a corrected version. Which corrected implementation is most appropriate?

○ A. Accept the AI's recommendation but add a second AES-128-CBC encryption pass with a different deterministic IV to increase the number of encryption layers

○ B. Replace AES-128-CBC with a stream cipher, since stream ciphers are not affected by IV reuse and eliminate the IV management problem entirely

○ C. Use AES-128-ECB to eliminate IV management complexity while retaining AES compatibility with existing tooling

● D. Use AES-256-GCM with a unique randomly generated nonce per encryption operation, store the encryption key in a dedicated secrets manager with role-based access control, and log all key retrieval events to an append-only audit trail

Correct answer: D

Why: The AI recommendation has three independent flaws: (1) a deterministic IV derived from the hostname is predictable and reused across encryption operations, violating the uniqueness requirement; (2) AES-128 provides a smaller key space than AES-256, AES-256-GCM is the current best practice for data-at-rest; (3) a plaintext environment variable readable by all processes is effectively an unauthenticated, broadly accessible key store. The corrected implementation addresses all three: random nonce per operation, key isolation via a secrets manager, and auditability. Adding more CBC layers (A) does not fix the deterministic IV or key exposure. Stream ciphers (B) are equally or more sensitive to nonce reuse than block ciphers in CTR/GCM mode. ECB (C) removes IV management by removing IV entirely, but reintroduces pattern leakage and is weaker than CBC.

*Why the others are wrong: **A**, Students who reason that "more encryption layers = more security" may select this, without recognising that the same fundamental flaws persist in each layer. **B**, This reflects a partial understanding: students who know that stream ciphers use nonces rather than IVs may believe this eliminates the reuse problem, when in fact nonce reuse in stream cipher modes (e.g., CTR) is catastrophically insecure. **C**, Students who focus narrowly on the IV management problem, and know that ECB has no IV, may select this as a simplification, without recognising that ECB's pattern leakage is a worse vulnerability than the IV problem they intended to fix.*